CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

CSPIT — Department of Computer Engineering · ITUE203 Web Development Frameworks

Student Name	Dhruv Bhoi	**Student ID**	**D26CE154**
Academic Year	2026–27 (ODD)	**Semester**	3rd Semester, B.Tech CE

Practical 14: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Practical Aim & Tools

Aim: Develop an Admin Dashboard with role-based access control, KPI metric cards, dynamic navigation, stored procedure monthly registration charts, user active/inactive toggles, and an audit trail viewer.

Tools & Technologies: PHP, MySQL Stored Procedures, Role-Based Access Control (RBAC), CSS Data Visualization

1. Enterprise Admin Features (`admin/dashboard.php` & `admin/audit.php`)

The administrative module provides executive-level monitoring of the entire StudentHub portal:
• KPI Stat Metric Cards: Real-time calculation of Total Students, Active Campus Events, Total RSVP Registrations, and Security Audit Actions.
• Stored Procedure Execution: Calls MySQL stored procedure `CALL sp_registrations_by_month()` to compute month-by-month registration velocity, rendering pure CSS proportional bar charts.
• User Lifecycle Controls: Interactive table displaying registered campus users with 1-click status toggles ('Active' ↔ 'Inactive') and last login timestamps.
• Security Audit Trail (`admin/audit.php`): Full system audit trail logging actions (`login`, `register`, `student_create`, `event_delete`), actor email, IP address, and timestamps.

2. Implementation Code: Stored Procedure & Dynamic Analytics

Listing: admin/dashboard.php — Stored Procedure Execution and User Lifecycle Management

```
// admin/dashboard.php - Stored Procedure Invocation & Analytics
require_once __DIR__ . '/../includes/functions.php';
require_once __DIR__ . '/../includes/db.php';
require_role('admin'); // Restricts access exclusively to administrators

// Execute Stored Procedure
$monthly = $pdo->query('CALL sp_registrations_by_month()')->fetchAll();
$pdo->query('SELECT 1')->closeCursor(); // Free connection cursor for subsequent queries

// User Active / Inactive Status Toggle
if ($_SERVER['REQUEST_METHOD'] === 'POST' && ($_POST['do'] ?? '') === 'toggle') {
```

```
    if (csrf_check()) {
        $uid = (int)$_POST['user_id'];
        $u = $pdo->prepare('SELECT status, email FROM users WHERE id=?');
        $u->execute([$uid]);
        $row = $u->fetch();
        $next = $row['status'] === 'active' ? 'inactive' : 'active';
        $pdo->prepare('UPDATE users SET status=? WHERE id=?')->execute([$next, $uid]);
        audit($pdo, 'user_status_' . $next, 'User: ' . $row['email']);
        flash('success', 'User account ' . $next . ' successfully.');
    }
    redirect('dashboard.php');
}
```

3. Verification Screenshot: Executive Admin Dashboard

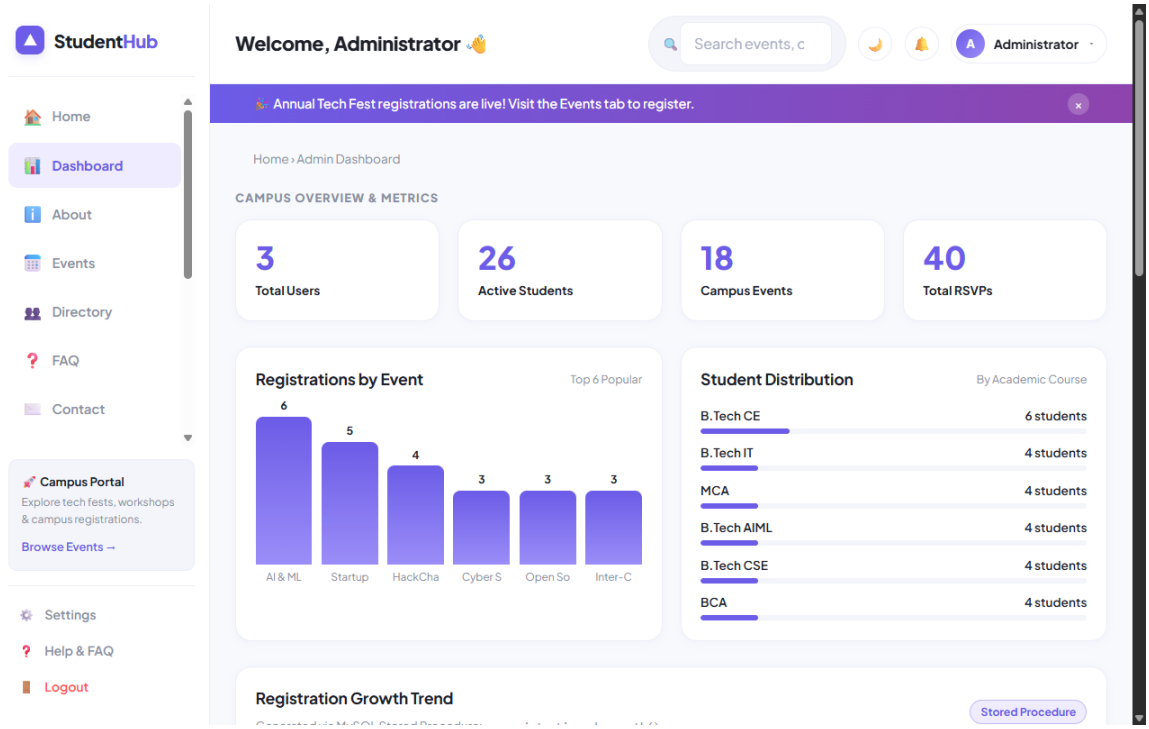


Figure: Admin Dashboard with KPI Stat Cards, Stored Procedure Monthly Chart, and User Status Toggles

4. Verification Screenshot: Security Audit Log Viewer

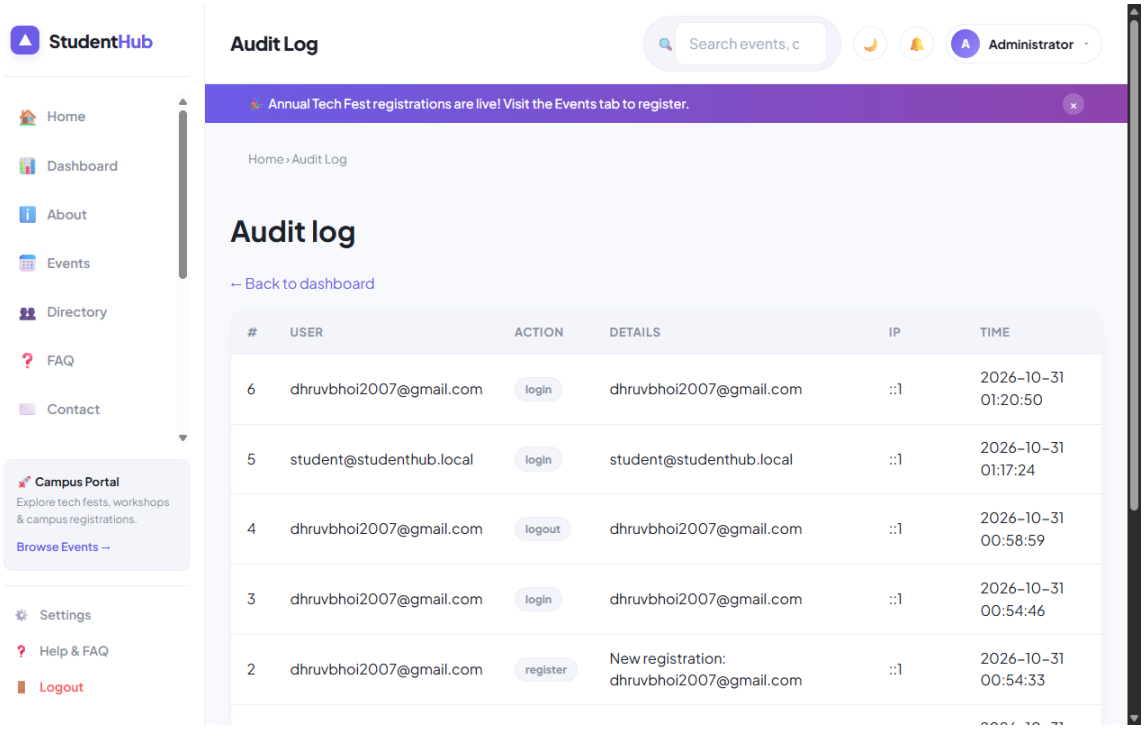


Figure: Immutable Security Audit Trail (admin/audit.php) Tracking User Actions and IP Addresses

5. Key Questions & Analysis

Q: How does `require_role('admin')` enforce access control across sensitive endpoints?

Ans: It checks `$_SESSION['user']['role'] === 'admin'`. If a student or guest attempts to access the script directly via URL, the function emits `HTTP 403 Forbidden`, dispatches an access denied flash message, and redirects the client to `dashboard.php`.

Q: Why are stored procedures used for aggregation queries like monthly registrations?

Ans: Stored procedures are pre-compiled and optimized on the MySQL database server. They minimize network latency by executing complex multi-table groupings internally, provide access abstraction, and enhance enterprise security.

Practical 14 Conclusion & Learning Outcomes

Developed an executive Admin Dashboard featuring real-time KPI metric calculation, stored procedure execution (sp_registrations_by_month) with CSS bar charts, user active/inactive toggles, and an immutable security audit trail (admin/audit.php).